WEEK 9 SESSION 1

Objects That Work Together

State validation, object references, composition and inheritance
BankAccount, SavingsAccount and AccountBook

Learning goals and lesson plan

Trace names, objects and changes to instance state.

Keep valid balances through constructors and methods.

Build an object that holds and uses other objects.

Compare inheritance with composition through running code.

Review 30 min + state rules 20 min + collaboration 20 min

Break 10 min + lab 30 min + review 10 min = 120 min

01. State and object identity

Two names can refer to one object

Week 7: class, object and instance

```
a = BankAccount("Minsu", 1000)
b = BankAccount("Jisu", 500)
```

Term	Meaning in this example
Class	BankAccount defines account behavior
Object	a and b refer to two separate account objects
Instance	Both account objects are instances of BankAccount

An object is already an instance; there is no extra conversion step.

Week 7: __init__ and self

```
class SimpleAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

account = SimpleAccount("Minsu", 1000)
```

__init__ initializes the newly created instance.

self receives the instance; owner and balance receive the other arguments.

This minimal recap has no validation; the main model adds it later.

Week 7: attributes and methods

```
class SimpleAccount:
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

account = SimpleAccount(1000)
account.deposit(200)
print(account.balance)  # 1200
```

Attributes store state; methods describe operations on the object.

Week 7: class and instance variables

```
class LabelledAccount:
    bank_name = "Python Bank"
    def __init__(self, owner):
        self.owner = owner

a = LabelledAccount("Minsu")
b = LabelledAccount("Jisu")
LabelledAccount.bank_name = "Campus Bank"
print(a.bank_name, b.bank_name)  # Campus Bank Campus Bank
```

bank_name belongs to the class; owner belongs to each instance.

Assigning a.bank_name instead would create an instance attribute here.

Week 7: encapsulation and abstraction

Principle	Focus	Bank-account example
Encapsulation	Related state and rules belong together	_balance and validated deposit/withdraw methods
Abstraction	Callers use a stable public interface	withdraw(amount) returns success or failure

One method can support both principles.

_balance is an internal-use convention, not an access lock.

Abstraction does not require an abstract base class.

Week 7: inheritance and polymorphism

```
accounts = [BankAccount("Minsu", 1000),
            SavingsAccount("Jisu", 500, 100)]
for account in accounts:
    print(account.account_type())
```

Principle	Where to look
Inheritance	SavingsAccount reuses BankAccount operations
Polymorphism	The same call yields Standard or Savings

The complete account classes are supplied in examples/models.py.

Week 7 retrieval check

1. Is Minsu’s account an object, an instance, or both?
2. What does __init__ initialize, and who receives self?
3. Which variable is per-instance: bank_name or owner?
4. Which principle keeps balance-change rules with the balance?
5. Which principle lets one caller use different account implementations?

Pair activity: answer first, then justify with one line of code.

We now apply these ideas to references and reliable state changes.

Week 7 recap: one class, two objects

```
a = BankAccount("Minsu", 1000)
b = BankAccount("Jisu", 500)
a.deposit(200)
print(a.get_balance())  # 1200
print(b.get_balance())  # 500
```

Each instance keeps its own owner and balance.

The same deposit method operates on the object passed as self.

Calling a method and passing self

```
a.deposit(200)
BankAccount.deposit(a, 200)
```

For this inherited-free call, both forms pass a as self.

Execute one form when you intend one deposit.

self is a conventional parameter name, not a Python keyword.

The method reads and changes that instance through self.

Assignment creates another reference

```
a = BankAccount("Minsu", 1000)
alias = a
alias.withdraw(300)
print(a.get_balance())  # 700
print(alias is a)       # True
```

Assignment binds another name to the existing object.

It does not construct a second BankAccount.

Same values, separate objects

```
a = BankAccount("Minsu", 1000)
b = BankAccount("Minsu", 1000)
a.withdraw(300)
print(a is b)           # False
print(b.get_balance())  # 1000
```

Calling the class twice creates separate instances here.

Equal-looking attributes do not imply shared identity.

Predict: which balance changes?

```
a = BankAccount("Minsu", 1000)
b = BankAccount("Jisu", 500)
c = a
accounts = [a, b]
accounts[0].withdraw(200)
print(c.get_balance(), b.get_balance())
```

Discuss for one minute, then trace each name.

What changes if accounts[0] is replaced with b afterward?

02. Valid state and method contracts

An accepted change keeps the account valid

The balance invariant

An invariant is a rule that should hold after construction and public operations.

Balance is a nonnegative integer in the smallest currency unit.

Deposits and withdrawals accept positive integers only.

A rejected operation changes nothing.

Internal _balance is updated through the account methods.

Which rule would balance -= amount violate if amount were negative?

Validate during initialization

```
class BankAccount:
    def __init__(self, owner, balance=0):
        if not isinstance(owner, str) or not owner.strip():
            raise ValueError("owner must be nonblank text")
        if type(balance) is not int or balance < 0:
            raise ValueError("balance must be a nonnegative integer")
        self.owner = owner.strip()
        self._balance = balance
```

Invalid starting data raises an exception before construction succeeds.

Constructor failure and caller handling

```
try:
    account = BankAccount("Minsu", -100)
except ValueError as error:
    print(error)
```

The assignment does not complete if construction raises.

The caller can explain the error or request another value.

Do not leave a successfully created account with an invalid balance.

A deposit that preserves the rules

```
def deposit(self, amount):
    if type(amount) is not int or amount <= 0:
        return False
    self._balance += amount
    return True
```

False means rejected and unchanged.

True means the balance has increased by amount.

Validate before mutating state.

A withdrawal with two kinds of failure

```
def withdraw(self, amount):
    if type(amount) is not int or amount <= 0:
        return False
    if amount > self._balance:
        return False
    self._balance -= amount
    return True
```

Invalid amount and insufficient funds both leave the balance unchanged.

Withdrawing the exact balance is allowed.

Trace state through a sequence

Operation, starting from 1000	Return	Balance afterward
deposit(200)	True	1200
withdraw(500)	True	700
withdraw(900)	False	700
withdraw(-50)	False	700
withdraw(700)	True	0

Read the return value and the new state together.

Encapsulation and public access

```
def get_balance(self):
    return self._balance
```

get_balance exposes a result without a balance-changing operation.

A leading underscore marks an internal-use convention.

Python does not prevent account._balance = -100.

Team code should respect the documented public interface.

Small checks for method contracts

```
a = BankAccount("Minsu", 1000)
assert a.withdraw(1200) is False
assert a.get_balance() == 1000
assert a.withdraw(1000) is True
assert a.get_balance() == 0
```

Check both success/failure and the resulting state.

Use assert for development checks; validate input with guards or exceptions.

03. Objects that collaborate

AccountBook has accounts; SavingsAccount is an account

Composition: an AccountBook holds accounts

```
class AccountBook:
    def __init__(self):
        self._accounts = []

    def add_account(self, account):
        if account in self._accounts:
            return False
        self._accounts.append(account)
        return True
```

A separate list belongs to each AccountBook instance.

The list stores references to account objects.

The shared mutable class-attribute trap

```
class BadBook:
    accounts = []
```

```
first = BadBook()
second = BadBook()
first.accounts.append("Minsu")
print(second.accounts)  # ["Minsu"]
```

Put per-book mutable state in self._accounts inside __init__.

Delegate to each account

```
def total_balance(self):
    total = 0
    for account in self._accounts:
        total += account.get_balance()
    return total
```

AccountBook coordinates accounts through their public methods.

It does not need to read account._balance.

An empty book returns 0.

A report uses the public interface

```
def report(self):
    rows = []
    for account in self._accounts:
        rows.append((account.owner, account.account_type(),
                     account.get_balance()))
    return rows
```

Return result data so the caller chooses how to display it.

The caller can print rows or save them later.

Do not return the mutable internal account list.

Changes remain visible through references

```
a = BankAccount("Minsu", 1000)
book = AccountBook()
book.add_account(a)
a.withdraw(300)
print(book.total_balance())  # 700
```

book and a refer to the same account object.

The book recalculates the total from current account state.

A separate empty AccountBook still has total 0.

Inheritance: a specialized account

```
class SavingsAccount(BankAccount):
    def __init__(self, owner, balance=0, bonus=100):
        super().__init__(owner, balance)
        if type(bonus) is not int or bonus <= 0:
            raise ValueError("bonus must be a positive integer")
        self.bonus = bonus

    def add_bonus(self):
        return self.deposit(self.bonus)
```

The subclass reuses the base account rules and adds one operation.

Overriding one public operation

```
class BankAccount:
    def account_type(self):
        return "Standard"

class SavingsAccount(BankAccount):
    def account_type(self):
        return "Savings"
```

A shared method name can provide type-specific behavior.

SavingsAccount still supports get_balance, deposit and withdraw.

One caller, different account types

```
accounts = [BankAccount("Minsu", 700),
            SavingsAccount("Jisu", 600, 100)]
for account in accounts:
    print(account.owner, account.account_type(), account.get_balance())
```

```
Minsu Standard 700
Jisu Savings 600
```

The caller uses the same operations on each object.

Inheritance and composition

Relationship	Example	Responsibility
Is an account	SavingsAccount inherits BankAccount	Preserve the account contract and extend behavior
Has accounts	AccountBook stores account references	Coordinate a collection of accounts

AccountBook should not inherit BankAccount just to reuse get_balance.

A dictionary and functions may be enough when no object state is needed.

Checkpoint: predict the combined result

```
a = BankAccount("Minsu", 1000)
b = SavingsAccount("Jisu", 500, 100)
book = AccountBook()
book.add_account(a)
book.add_account(b)
a.withdraw(300)
b.add_bonus()
print(book.total_balance())
```

What happens to the total if a.withdraw(900) is called next?

04. Lab: complete the account model

10-minute break, then 30 minutes of Python practice

Files and first run

```
python 9week/examples/session1/accounts.py
python 9week/labs/session1/accounts.py
```

Python 3.10+; no external packages or data downloads.

The demo runs; the starter raises NotImplementedError at a TODO.

Complete withdraw, total_balance and report.

report returns a list of (owner, type, balance) tuples.

Lab A: preserve the balance invariant

10 minutes: implement BankAccount.withdraw(amount).

Reject nonintegers, zero, negatives and overdrafts.

Reject True as an amount under this lesson's integer policy.

Rejected calls return False and preserve the old balance.

Accepted calls subtract amount and return True.

Test a withdrawal equal to the available balance.

Lab B: coordinate existing objects

8 minutes: implement AccountBook.total_balance() and report().

Read account data through get_balance() and account_type().

Empty book: total 0, report [].

Preserve insertion order in report.

Keep separate books independent.

Change an account after adding it and recompute the total.

Lab C: check the full demo

```
Withdraw: True 700
Rejected: False 700
Bonus: True 600
Minsu Standard 700
Jisu Savings 600
Total: 1300
```

7 minutes: compare output and add checks for failure cases.

Submit accounts.py and a short explanation of one state trace.

Review and Session 2

1. Why does alias.withdraw change what a.get_balance returns?
2. Why does an invalid withdrawal need a state check as well as False?
3. Why must each book create its own list in __init__?
4. Which relationship uses inheritance, and which uses composition?

Session 2: choose project objects, data and function contracts.

Submit the completed script and explain one design choice.